

SC2005 Lab 2 — Processes, Scheduling, and User Threads

What this lab is about

In this lab, you will examine how xv6 represents and schedules processes, modify its scheduler, and complete a small user-level threading library. The exercises connect process state, preemption, kernel context switching, and cooperative user-thread switching.

This is an AI-assisted lab. You may use the course Gemini Notebook, which contains the Lab 2 source tree and the xv6 book. Use it to navigate and explain the supplied sources; it does not replace reading the cited code, making your own implementation, or validating behavior in xv6.

By the end of the lab, you should be able to:

1. Locate the process state and context maintained by xv6 and explain why user code cannot directly read the kernel process table.
2. Follow a short guided trace from `fork()` to the first scheduling of a child through `forkret()`, the function every newly created process first returns through.
3. Explain how a timer interrupt can trigger preemption, using the path through `yield()`, `sched()`, `scheduler()`, and `swtch()`; these functions also serve scheduling paths that are not timer-triggered.
4. Implement strict priority for runnable even-PID processes and explain when odd-PID processes can starve.
5. Explain which register state a context switch preserves and why callee-saved registers are sufficient.
6. Complete and debug the supplied cooperative user-level threading library.

How to use Gemini Notebook in this lab

Prompt convention: text headed “Ask Gemini Notebook” inside a pale-blue shaded block is a prompt to give Gemini Notebook. All other text is an instruction, explanation, specification, command, or checkpoint for you. In particular, the required policy in §2.4 is an instruction to implement.

Use the same evidence loop at each stage:

1. **Formulate expectations:** write down what you expect before asking AI or running the program.
2. **Research:** ask a focused question about the supplied source and request exact paths, functions, and line ranges.
3. **Verify in source:** open every cited location and correct claims that do not match this Lab 2 branch.
4. **Verify by execution:** build and run a controlled test that can confirm or reject the claim.
5. **Explain:** state the result and its limitation in your own words.

Ask Gemini Notebook

In the supplied Lab 2 xv6 source, help me investigate **[specific behavior]**. Give the exact original file paths, functions or symbols, and line ranges that support each claim. Distinguish source evidence from inference and cite the relevant xv6 book section. Guide me with questions and a verification checklist; do not provide a complete solution or patch.

Do not ask Gemini Notebook to inspect the code you write. When debugging, describe the exact compiler error, runtime symptom, test output, or debugger observation, then use the response as a checklist to inspect your own work.

Use the prompt blocks when you need help; you do not need to use every prompt or submit a transcript of each AI exchange. Keep the evidence loop above. Annotated source locations, saved terminal output, and a brief explanation are sufficient checkpoint evidence; you do not need separate trace or design tables.

At every checkpoint, be ready to show:

1. your prediction or design invariant;
2. the supplied-source evidence you verified;
3. the command and output used for testing; and
4. a short explanation in your own words.

Before you begin: confirm the checkout and baseline (indicative: 5 minutes)

This guide uses the same prompt labels as Lab 1:

Label used in this guide	Where the command runs	Examples
host\$	The Linux lab machine or VM	git, make qemu, gdb-multiarch
xv6\$	The xv6 shell running inside QEMU	ls, forktest, uthread_test

Clone Github repository you will use for this lab session:

```
host$ git clone https://github.com/CPS-research-group/ntu-sc2005-xv6.git lab2-xv6
host$ cd lab2-xv6
host$ git checkout lab2
host$ git branch --show-current
host$ make clean
```

Confirm the active branch is `lab2`, then boot the prepared checkout with one CPU. Keep this session for Part 1 and the §2.1 baseline. If setup fails, ask the instructor for the prepared environment.

```
host$ make CPUS=1 qemu
```

If the clean build reaches the xv6 shell, establish the unmodified threading baseline:

```
xv6$ uthread_test
```

Save the build result and the unmodified `uthread_test` output for comparison in §3.5. Source inspection predicts a silent return to the shell because the switch call is still commented out; this is not a recorded run. Keep xv6 running for Part 1.

Ask Gemini Notebook

Using only the supplied sources, help me locate the files for process state, timer preemption, kernel context switching, and the user-thread TODOs. Include `user/schedtest.c` and `user/starvetest.c` from the supplied workload packet and their `Makefile` build entries. Give exact paths and symbols, and identify missing files rather than inventing them. Do not provide implementations.

Checkpoint 0: orient yourself before editing

You are ready to continue when you can:

1. show that the active branch is `lab2`;
2. show the unmodified build result and, if it builds, the `uthread_test` behavior;
3. identify the source files used in each of the three main parts of this lab; and
4. explain why a clean build was appropriate after the branch change.

Part 1 — Trace processes in xv6 (indicative: 15 minutes)

In `kernel/proc.h`, `struct proc` holds per-process metadata, an embedded `struct context` for saved kernel registers, and a pointer to a separately allocated `struct trapframe` for user registers on the trap/return path. `kernel/proc.c` defines the `proc[]` table. The user address space is the memory mapped by the process's user page table. In this Part, locate these distinct kinds of state, follow a child to its first scheduling, and inspect the process table with `Ctrl-p`.

1.1 Locate process state and ownership

Locate `enum procstate` and `struct proc` in `kernel/proc.h`, then find `proc[]` in `kernel/proc.c`. Focus on three transitions you will use in Part 2: a new child becomes `RUNNABLE`, `scheduler()` changes `RUNNABLE` to `RUNNING`, and `yield()` changes `RUNNING` back to `RUNNABLE`. Record the assignment and enclosing function for each; you do not need to enumerate every lifecycle transition.

Ask Gemini Notebook

Help me locate where a child becomes `RUNNABLE`, where `scheduler()` makes it `RUNNING`, and where `yield()` makes it `RUNNABLE` again. Give source locations and ask me to explain which code owns each transition. Do not build a complete lifecycle table.

Verify the three assignments in the source and explain why only a `RUNNABLE` entry may be selected by the scheduler.

1.2 Trace a newly forked command

Use `ls` as the example: the shell forks a child that later executes the program. Follow this guided route in the supplied code: `user/sh.c` → `fork()` and `allocproc()` in `kernel/proc.c` → `scheduler()` and `swtch()` → `forkret()`. Pay special attention to the comment above `forkret()` in `kernel/proc.c`.

Ask Gemini Notebook

Along the guided `fork-to-forkret` route, help me locate the new child's saved context setup, its `RUNNABLE` assignment, and its first context restore. Ask me what the comment above `forkret()` explains. Give exact source locations; do not provide a full syscall trace or the answer to the comment question.

Annotate the saved-context setup and first-restore locations. Explain in one or two sentences how the child reaches `forkret()` on its first scheduling; no full trace table is required.

Use your annotations to distinguish the saved kernel context from the `trapframe` and user address space introduced above.

1.3 Observe the process table through `Ctrl-p`

At the idle xv6 shell, press `Ctrl-p` and save the PID, state, and name columns. If no `RUNNABLE` entry appears, keep that result: you will compare it with the busy workload in §2.5.

Inspect the `Ctrl-p` handling in `kernel/console.c` and the `procdump()` implementation in `kernel/proc.c`.

Ask Gemini Notebook

Locate the `Ctrl-p` handler and `procdump()`. Which process fields are printed, and why can this kernel debugging path read `proc[]` while a user program cannot? Point me to the source and the comment about locking.

Checkpoint 1: explain process state and visibility

You are ready when you can:

1. show the three state-transition assignments and your short guided trace to `forkret()`;
2. distinguish a `trapframe`, a saved kernel context, and a user address space;
3. show your initial `Ctrl-p` listing and identify its PID, state, and name columns;
4. locate the `Ctrl-p` call to `procdump()` and state why its listing is only a debugging snapshot; and
5. explain why an ordinary user program cannot directly dereference `proc[]`.

Part 2 — Modify scheduling and reason about context switching (indicative: 35 minutes)

Carry your three §1.1 state transitions and saved `Ctrl-p` listing into this Part. A `RUNNABLE` entry (printed as “`runble`”) is eligible for selection; your scheduler must preserve the locked `RUNNABLE` → `RUNNING` transition. In §2.5 you will compare the idle listing with a supplied busy workload.

In this `lab2` branch, each CPU scans `proc[]` in slot order and switches to entries it finds `RUNNABLE`. After `swtch()` returns, the inner scan continues to the next slot; only a new full pass starts at `proc[0]`. Timer interrupts can cause a running process to yield. Use `yield()`, `sched()`, `scheduler()`, and `swtch()` to connect this behavior to the context switch, then implement and test strict even-PID priority.

2.1 Observe default xv6 scheduler behavior

Use `CPUS=1` for both baseline and modified runs. Reuse your initial session if it already uses one CPU; otherwise exit QEMU with `Ctrl-a x` and use the command below. Save the original `scheduler()` before editing. Keep the complete baseline output and command for the side-by-side comparison in §2.5.

```
host$ make CPUS=1 qemu
```

Read the supplied `user/schedtest.c`. It creates four children, releases them through a pipe gate, and gives each child four finite CPU-work rounds with printing between rounds. Each round performs 4,000,000 loop iterations. The children do not call `sleep()`; the parent calls `sleep(1)` between child creations. `ROUNDS` is a compile-time constant, not a command-line option. Gate reads and output can block, so do not assume every child stays `RUNNABLE` for the entire test.

```
xv6$ schedtest
```

Run `schedtest` once under the stock scheduler and save the complete output with its actual PIDs. Because the children eventually exit, this baseline can show postponement and later progress; use the separate §2.5 workload to reason about indefinite starvation.

Do not infer a fairness guarantee or a precise scheduling sequence from print order alone.

2.2 Trace preemption and the scheduler loop

Inspect the timer-interrupt paths in `kernel/trap.c`, then follow `yield()`, `sched()`, and `scheduler()` in `kernel/proc.c`. Inspect `struct context` in `kernel/proc.h` and the register save/restore sequence in `kernel/swtch.S`.

Ask Gemini Notebook

Help me follow one timer-triggered preemption to `yield()`, `sched()`, and the per-CPU scheduler. Point out the two `swtch()` call sites, which contexts they save and restore, and the lock/state checks I must preserve. Give source locations and questions, not a completed trace table.

Annotate the two `switch()` call sites in `kernel/proc.c`: identify the context saved and restored at each. Then explain the preemption path aloud or in a few lines, including the `RUNNING` → `RUNNABLE` change and where a suspended process later resumes.

2.3 Explain the callee-saved register rule

Prerequisite refresher: revisit CPU registers, function calls, and stacks from SC1006, Computer Organisation & Architecture. The ABI register names refer to CPU registers: `ra` (`x1`) is the return address, `sp` (`x2`) is the stack pointer, and `s0–s11` (including `s0/x8` and `s1/x9`) are saved registers. “Caller-saved” means the caller must protect values it needs across a call; “callee-saved” means the called function must preserve them. Use Table 1, “Integer register convention,” in §1.1 of the RISC-V ABIs Specification to classify the registers. The table does not answer why `switch` saves its particular set; that is your task below.

Compare the fields of `struct context` with the loads and stores in `kernel/switch.S`. Then answer the design question: why can `switch()` omit the general caller-saved register set while saving the callee-saved state and the return address? Your explanation must use the function-calling convention and account for both `ra` and `sp`; do not assume `ra` is callee-saved.

Ask Gemini Notebook

Use the supplied `struct context`, `kernel/switch.S`, and the xv6 book to help me reason about which RISC-V registers cross a call to `switch()`. Ask me to classify caller-saved and callee-saved state, and help me verify how `ra` and `sp` allow execution to resume. Do not simply give me the final answer.

2.4 Define strict even-PID priority

Modify `scheduler()` with this required policy:

At each CPU's scheduling selection point, select a `RUNNABLE` process with an even PID whenever an eligible even-PID process is found. Select an odd-PID `RUNNABLE` process only after the scheduler has found no eligible even-PID process. This is strict priority between the two PID classes; the existing process-table order may remain the order within each class.

Preserve the existing locking discipline, state transition to `RUNNING`, assignment to `c->proc`, call to `switch()`, and cleanup after the process returns. Do not remove timer preemption.

Ask Gemini Notebook

Inspect the supplied `scheduler()` and help me plan a strict two-class selection policy: runnable even PIDs first, odd PIDs only when no runnable even PID is eligible. Identify the invariants around `p->lock`, `p->state`, `c->proc`, and `switch()`, and list race or starvation questions I should test. Give pseudocode-level control flow and exact source citations, but no C patch.

Verify the plan against `kernel/proc.c`, then implement it yourself.

2.5 Run the supplied test and analyze starvation

Run the same `schedtest` workload with your modified scheduler and `CPUS=1`. Place the saved §2.1 output beside the new output. Explain differences using actual PIDs and runnable intervals, not a memorized line sequence.

Run xv6 with one CPU:

```
host$ make CPUS=1 qemu
xv6$ schedtest
```

Next, read the supplied `user/starvetest.c`; you do not need to design or implement another program. It creates two children, checks that one PID is even and the other odd, waits for their readiness signals, and releases them through a pipe gate. After the gate, each child enters an endless CPU loop without sleep, I/O, or exit. The parent prints the child PIDs before release and then waits. Predict which child will receive the CPU under your policy, then run:

```
xv6$ starvetest
```

Press `Ctrl-p` twice during the run and save the listings. Use the printed child PIDs to distinguish the children from the parent. Look for the odd child repeatedly listed as “`runble`” while the even child runs; it may be printed as “`run`” when the console interrupt arrives. Compare with §1.3. These snapshots show postponement over the observation interval. Explain from the policy why it can continue indefinitely if the even child never blocks or exits.

Stop the observation with QEMU’s `Ctrl-a x` sequence, then restart QEMU. The xv6 shell may itself be starved; do not rely on a shell kill command. Keep the test at `CPUS=1`: an even process already `RUNNING` on another CPU is not a `RUNNABLE` candidate for this CPU, so a single busy even process does not establish starvation across three CPUs.

Ask Gemini Notebook

My strict-priority scheduler [fails to boot / hangs / violates the PID policy / breaks a test]. The exact symptom is: _____. Based only on the supplied scheduler, locking, trap, and context-switch code, give me a ranked diagnostic checklist. Focus on lock lifetime, scan structure, process state, `c->proc`, and whether the test actually keeps both PID classes runnable. Do not provide a patch.

Optional extension: First-Come, First-Served (will not be evaluated)

After saving a working strict-priority version, optionally design an FCFS scheduler that selects the oldest `RUNNABLE` process. Define when creation time is assigned, how ties are broken, how the field is protected, and whether preemption is retained. Return to your saved strict even-PID version for the Part 2 evidence; then follow the stock-scheduler restoration step below before Part 3 and final verification.

Before continuing to Part 3 (required)

Before leaving Part 2, save your strict-priority code, both `schedtest` outputs, and the `starvetest Ctrl-p` evidence. Restore only `scheduler()` to the original `lab2` implementation for Part 3 and final verification; retain your other changes and keep the strict-priority version for assessment. This avoids imposing strict priority on tests that assume

eventual progress. The full `forktest` and `usertests` regressions are instructor preparation, not required student runs in the two-hour core.

Checkpoint 2: explain policy and mechanism

You are ready when you can:

1. explain one timer-triggered preemption using your two annotated `swtch()` call sites;
2. distinguish the per-process context from the per-CPU scheduler context;
3. explain why the context contains callee-saved registers, `ra`, and `sp`;
4. state the strict even-PID policy and compare the saved `schedtest` baseline and modified outputs;
5. show that your strict-priority code is saved and the working scheduler is restored for Part 3; and
6. use `Ctrl-p` evidence to describe observed odd-PID postponement, and argue from the policy when that postponement can continue indefinitely.

Part 3 — Complete the user-level threading library (indicative: 45 minutes)

Carry this question from Part 2: a timer can interrupt user execution before the kernel switches processes, so that path uses both a `trapframe` and a saved kernel context. Why does a cooperative switch at a user-level function call need only the library's saved context, without creating another `trapframe` for each user thread? Keep the stock scheduler restored for this Part and Part 4; retain your saved strict-priority version for Part 2 assessment.

Complete the supplied cooperative threading library inside one xv6 process. The kernel schedules that process; the library switches threads when they call `thread_yield()` or `thread_exit()`. The threads share an address space, but each needs its own stack and saved context. Your C layout and assembly offsets must agree, and both new and suspended threads must resume correctly.

3.1 Inspect the unfinished library

Read these files together:

1. `user/uthread.c`: thread states, stacks, selection logic, and unfinished context setup;
2. `user/uthread_switch.S`: the unfinished register save/restore routine;
3. `user/uthread_test.c`: three test functions and their yield/exit behavior; and
4. `kernel/swtch.S` plus `struct context`: a kernel example of a RISC-V context switch.

Do not copy the kernel implementation blindly. Determine what representation the user library needs and ensure that the C layout and assembly offsets agree exactly.

3.2 Design the saved thread context

For your stack design, use the standard RISC-V ABI requirement: `sp` must be aligned to 16 bytes on function entry and remain aligned during standard-ABI execution (§2.1, “Integer Calling Convention,” in the RISC-V ABIs Specification). Check that your chosen thread representation satisfies this; “consider alignment” is not a separate unstated requirement.

Ask Gemini Notebook

Help me check three design choices: which register fields and assembly offsets must agree, how a new thread's `ra` and `sp` differ from a resumed thread's, and how to satisfy 16-byte stack alignment. Use the supplied source and ABI reference to guide my reasoning; do not provide C or assembly.

Before editing, make a brief design check: (1) register field order agrees with every assembly offset; (2) the initial `ra` and `sp` let a new thread enter its function on its own stack; (3) the stack pointer satisfies the 16-byte alignment rule. Annotate your proposed layout and explain these choices; no separate design table is required.

3.3 Plan the first execution of a new thread

A resumed thread returns to an address restored by `thread_switch`, using the restored stack pointer. A new thread has never previously called the switch routine, so `thread_create()` must construct a context that makes its first restore behave correctly.

Hint: Check `kernel/switch.S` plus `struct context` for their implementation.

Ask Gemini Notebook

Reason from the final `ret` in `user/uthread_switch.S` and the call sites in `user/uthread.c`: when a never-run thread is selected, what saved state must cause its function to begin on its own stack? How do I satisfy the 16-byte stack-alignment rule from §3.2, and what function-return behavior must I consider? Guide me through the control flow and source evidence without giving implementation code.

3.4 Implement one verified component at a time

Complete the supplied `TODO` locations in this order:

1. add a saved context representation to `struct thread`;
2. initialize a new thread's context and stack in `thread_create()`;
3. pass the outgoing and incoming contexts from `thread_schedule()`; and
4. save and restore the agreed register layout in `thread_switch`.

Build after each coherent change. A C structure whose field order does not match the assembly offsets may compile successfully and still fail at runtime.

Ask Gemini Notebook

My `uthread_test` [does not build / prints nothing / starts one thread only / crashes / repeats or skips values]. The exact first symptom is: _____. Give me a diagnostic checklist based only on the supplied user-thread and kernel switch examples. Prioritize C/assembly layout agreement, switch arguments, initial `ra` and `sp`, stack alignment, and thread-state transitions. Do not provide a solution or patch.

3.5 Verify the complete behavior

Run the completed test once here and save the complete output. Run it again during final verification to check repeatability:

```
xv6$ uthread_test
```

The three threads should start, each should print values from 0 through 99 while yielding to the others, and each should report that it exits after 100 iterations. No thread should be skipped, repeat a count, or continue after exiting. The supplied scheduler finally reports that no runnable user threads remain. Explain the order from the startup waits and thread-selection code; do not assume it must match the `thread_create()` call order.

The following are source-defined output checks, not a captured transcript or a required interleaving. Check your complete run for all of them:

```
thread_a started; thread_b started; thread_c started
```

Each thread prints its own values 0 through 99 exactly once.

`thread_a: exit after 100; thread_b: exit after 100; thread_c: exit after 100`

Use the startup waits and your runnable-thread scan to explain why counting order can differ from start order.

`thread_schedule: no runnable threads`

Compare the completed run with your unmodified baseline: what changed, and which TODOs enabled it? Use the build result, thread counts, yields, and state transitions as evidence. If the baseline failed to build, compare that diagnostic first. Keep this brief explanation with your output.

Checkpoint 3: explain the user-thread switch

You are ready when you can:

1. distinguish an xv6 process, its kernel thread, and the cooperative user threads inside it;
2. show that every context field matches the corresponding assembly offset;
3. explain how a new thread begins its function on its own stack;
4. trace one `thread_yield()` from the outgoing thread to the resumed thread; and
5. show one complete `uthread_test` run, compare it with the baseline, and confirm a repeat run during final verification.

Part 4 — Debug a thread switch with GDB (optional; will not be evaluated; indicative: 25 minutes)

After completing the switch in Part 3, you can use GDB to inspect the saved values directly; this Part is optional and is not examinable. Checkpoint 4 is a self-assessment aid only. The six final questions assess Parts 1–3 and do not require GDB commands or Part 4 material.

Use GDB when the runtime symptom does not reveal which saved value is wrong. In Part 4, you will use GDB to observe a user-level context switch rather than reasoning only from program output. You will identify the outgoing and incoming contexts, track changes to `ra` and `sp`, and diagnose errors involving initial context setup, stack layout, switch arguments, or C/assembly offsets. Prefer function breakpoints because editing your code can move source line numbers.

4.1 Start a two-terminal debugging session

In the first host terminal, start xv6 paused for a debugger:

```
host$ make qemu-gdb
```

In a second host terminal, change to the same xv6 source directory before starting the debugger:

```
host$ gdb-multiarch
```

The `Makefile` generates a project `.gdbinit` from `.gdbinit.tmpl-riscv`. Auto-loading that project file supplies the connection to QEMU. If GDB declines to load it, follow these steps:

1. Read GDB's warning and copy the exact `add-auto-load-safe-path` command it suggests for this checkout's `.gdbinit`.
2. Quit GDB. Open the personal GDB initialization file named in the warning (create it if needed) and add that command there. Do not put it in the project `.gdbinit` that GDB has just declined to load.
3. Restart `gdb-multiarch` from the xv6 source directory. Confirm that it loads the project `.gdbinit` and connects to the waiting QEMU instance before continuing below.

```
(gdb) file user/_uthread_test
```

```
(gdb) break thread_schedule
```

```
(gdb) break thread_switch
```

```
(gdb) continue
```

When the xv6 shell appears, run `uthread_test`. A breakpoint at a user virtual address can sometimes be encountered while another user process occupies the same address; continue until the stopped execution corresponds to `uthread_test`.

4.2 Inspect one switch

At `thread_schedule`, step until the outgoing and incoming thread pointers are valid, then inspect them. At `thread_switch`, single-step the assembly and compare registers with the memory locations used by your context layout:

```
(gdb) print/x *current_thread
(gdb) print/x *next_thread
(gdb) info registers ra sp s0 s1
(gdb) x/16gx next_thread->stack
(gdb) stepi
```

Adjust expressions to match the fields you created. Record the old and new `ra` and `sp`, the relevant context addresses, and the instruction reached after the switch.

Ask Gemini Notebook

I stopped immediately before and after one `thread_switch`. My expected control flow was ____, but the observed `ra`, `sp`, context addresses, or next instruction were ____. Using only the supplied source and xv6 book, give me questions and checks that distinguish a bad initial context, a C/assembly offset mismatch, a bad switch argument, and a state-transition bug. Do not write code.

Online GDB documentation: <https://sourceware.org/gdb/current/onlinedocs/gdb.html>.

Checkpoint 4: self-assessment only — explain a switch from evidence

For your own self-assessment, check whether you can:

1. stop at `thread_switch` using a function breakpoint, which does not depend on source line numbers staying unchanged;
2. identify the outgoing and incoming saved-context locations;
3. show how `ra` and `sp` change across one switch; and
4. explain why `ret` resumes a different thread rather than returning normally to the outgoing one.

Final verification and checkpoint (indicative: 10 minutes)

Exit QEMU. Confirm that your strict even-PID scheduler and its Part 2 evidence are saved, and that `scheduler()` in the working tree is the original `lab2` implementation, as instructed at the end of Part 2. Keep your completed user-thread changes. Use `CPUS=3` explicitly for this final clean build; this CPU setting alone does not restore the stock scheduler.

```
host$ make clean
```

```
host$ make CPUS=3 qemu
```

For the timed completion check, repeat `uthread_test` and compare it with your saved §3.5 run:

```
xv6$ uthread_test
```

Keep your Part 2 scheduler code and evidence, completed user-thread code, baseline comparison, and final output. Your instructor validates the reference implementation with `forktest` and `usertests` before class. You may run these additional regressions after the core work with the stock scheduler and `CPUS=3`; allow extra time. They are not required for the two-hour completion checkpoint.

Use the following six questions as a short self-check before leaving. You should be able to answer them without AI, using the evidence already collected; no separate written response set is required. They cover Parts 1–3 and exclude GDB and optional Part 4 material:

1. Why can `procdump()` read `proc[ ]` while a normal user program cannot?
2. How does a timer interrupt lead from a running process to the scheduler and then back into a process?
3. Under the required strict-priority policy, what workload can starve an odd-PID process?
4. Why are callee-saved registers, `ra`, and `sp` sufficient for the supplied cooperative switch?
5. How does a never-run user thread begin at its function on its own stack?
6. What is the difference between the kernel scheduling an xv6 process and the library scheduling one of its user threads?

References

1. SC2005 xv6 Lab 2 branch: <https://github.com/CPS-research-group/ntu-sc2005-xv6/tree/lab2>.
2. Russ Cox, Frans Kaashoek, and Robert Morris, [xv6: a simple, Unix-like teaching operating system](#), especially §§1.1, 2.5, and 7.1–7.3.
3. [MIT 6.1810 Lab: Multithreading](#), especially the user-level thread-switch exercise.
4. SC2005 Process Scheduling lecture.
5. RISC-V ABIs Specification, §1.1, Table 1 (“Integer register convention”), and §2.1 (“Integer Calling Convention”): <https://riscv-non-isa.github.io/riscv-elf-psabi-doc/>.